

Lastenheft: Desktop Drum-Programming-Software

Status: Entwurf / Planungsgrundlage

Stand: 03.10.2026 (abgestimmt mit Pflichtenheft v1.0)

Projekttyp: Nebenprojekt (nicht hauptberuflich)

1. Projektübersicht

1.1 Zielsetzung

Entwicklung einer plattformübergreifenden Desktop-Anwendung zur Programmierung von Schlagzeug-Patterns (Drum Programming). Die Software soll Echtzeit-Eingabe über Computertastatur und MIDI-Hardware sowie klassische Maus-Editierung in einem Pattern-Editor kombinieren.

1.2 Auftraggeber / Entwickler

Einzelentwickler, Hobby-/Nebenprojekt neben Hauptberuf.

1.3 Zeitrahmen

- ca. 9–12 Std./Woche (3 Std. × 3–4 Tage)
 - Geschätzte Gesamtdauer: ca. 16 Wochen bis MVP, ca. 28 Wochen (ca. 6,5 Monate) bis Version 1.0 (Detailplanung im Pflichtenheft, Kapitel 8 und 9)
-

2. Technischer Stack

Bereich	Wahl	Begründung
Framework	JUCE (C++)	Industriestandard für Audio-Anwendungen, deckt Audio-I/O, MIDI, GUI in einem Framework ab, AGPLv3 = Open Source nutzbar
Sprache	C++	Notwendig für JUCE; Umstieg von C#/Delphi mit überschaubarer Lernkurve
IDE	CLion	Beste CMake-Integration, Debugger/Refactoring-Komfort vergleichbar mit Delphi/Visual Studio
Build-System	CMake	JUCE-Standard seit einigen Jahren, CI-fähig (z. B. GitHub Actions für Windows-Build)
Versionierung	Git	—

3. Funktionale Anforderungen

3.1 Audio-Eingabe / Sample-Wiedergabe

- ✓ **Entscheidung:** MVP unterstützt **WAV**; Architektur über JUCEs `AudioFormatManager` so anlegen, dass weitere Formate (AIFF, FLAC) später ohne Umbau der Sample-Engine ergänzbar sind (JUCE bringt Reader für beide bereits mit)
- ☐ Laden und Abspielen mehrerer Audio-Samples
- ☐ Polyphone Wiedergabe (mehrere Samples gleichzeitig, überlappend)
- ✓ **Entscheidung:** Zuordnung von Samples zu Pads/Trigger-Slots nach **General-MIDI-Standard-Drum-Map** (Kanal 10), frei überschreibbar
 - Default-Mapping garantiert MIDI-Import/Export-Kompatibilität ohne Konvertierung (Anforderung 3.4)
 - Kernbereich Noten 35–59 (Kick, Snare, Toms, Hi-Hats, Crash/Ride, Cowbell, Clap etc.) — siehe Tabelle unten
 - Pro Sample-Slot individuell überschreibbar für eigene/untypische Percussion-Samples
- ✓ **Entscheidung (30.09.2026): Pitch-Regler je Sample-Slot** (± 12 Halbtöne) als einfache Tonhöhenänderung über die Abspielgeschwindigkeit (Sample wird dabei kürzer/länger); Priorität Soll. Gilt nicht als Effekt im Sinne von Abschnitt 8

MIDI-Note	Instrument	MIDI-Note	Instrument
35	Acoustic Bass Drum	48	Hi-Mid Tom
36	Bass Drum 1 (Kick)	49	Crash Cymbal 1
37	Side Stick	50	High Tom
38	Acoustic Snare	51	Ride Cymbal 1
39	Hand Clap	52	Chinese Cymbal
40	Electric Snare	53	Ride Bell
41	Low Floor Tom	54	Tambourine
42	Closed Hi-Hat	55	Splash Cymbal
43	High Floor Tom	56	Cowbell
44	Pedal Hi-Hat	57	Crash Cymbal 2
45	Low Tom	58	Vibraslap
46	Open Hi-Hat	59	Ride Cymbal 2
47	Low-Mid Tom		

3.2 Echtzeit-Eingabe

- ✓ **Fokus:** manuelle Drum-Eingabe über Computertastatur und MIDI-Hardware-Geräte (nicht automatische Pattern-Generierung)
- ☐ Eingabe über Computertastatur (Tasten-Mapping auf Sample-Slots)
- ✓ **Entscheidung (30.09.2026):** Tasten-Mapping über **physische Tastenpositionen (Scancodes)**, damit es auf QWERTZ und QWERTY gleich liegt, und **im GUI frei einstellbar** (MVP)
 - Taste je Slot im Inspector sowie Dialog „Tastatur-Mapping“ mit Lernmodus (Feld anklicken, Taste drücken) und „Standard wiederherstellen“
 - Doppelbelegungen werden gemeldet
 - Mapping wird als Programmeinstellung gespeichert und gilt für alle Projekte
- ☐ Eingabe über MIDI-Hardware-Controller (Note-On/Off-Events empfangen)

- ✓ **Entscheidung (30.09.2026): Aufnahme der Live-Eingabe ins Pattern ist Kernfunktion und Teil des MVP**
 - Rec + Play nimmt Anschläge von Tastatur und MIDI-Hardware mit Velocity unquantisiert im Loop ins aktive Pattern auf
 - Metronom, Vorzähler (0-2 Takte) und Latenzkompensation gehören dazu
 - Aufnahme-Modi Overdub (Standard) und Ersetzen; jeder Durchgang ist ein Undo-Schritt
- ✓ **Entscheidung:** Echtzeit-Quantisierung/Swing-Funktion **nicht Teil des MVP** — spätere Phase, nach Grundfunktionen (Sample-Wiedergabe, Eingabe, Piano-Roll, Song-Arrangement)

3.3 Editierung

- ✓ **Entscheidung:** Piano-Roll als primäre Editier-Ansicht (freie Notenposition, -länge, Velocity), mit optionalem **Snap-to-Grid**
 - Datenmodell speichert Noten intern immer in absoluten Zeitwerten (Ticks/Samples) — Raster ist reine Editier-Hilfe, keine Datenstruktur-Einschränkung
 - **Entscheidung:** Zeitauflösung **960 PPQ** wie in Reaper ($1/16 = 240$ Ticks, $1/32$ -Triole = 80 Ticks)
 - Snap an/aus umschaltbar (Toolbar-Toggle), Raster-Auflösung wählbar ($1/4$, $1/8$, $1/16$, $1/32$, Triolen)
 - Modifier-Taste (z. B. Alt) erlaubt temporär freies Positionieren trotz aktivem Snap
 - Ermöglicht nachträgliches Quantisieren von per MIDI-Hardware frei eingespielten Noten, statt Timing beim Einspielen zu zerstören
- ☐ Noten/Hits hinzufügen, verschieben, löschen per Mausklick
- ☐ Pattern-Länge/Taktart einstellbar

3.4 Song-Arrangement

- ✓ **Entscheidung:** Zwei-Ebenen-Modell — Pattern-Ebene (einzelne Patterns im Piano-Roll) + Song-Ebene (Verkettung zu vollständigem Song)
 - Song-Track: horizontale Timeline-Ansicht, Pattern-Blöcke per Drag&Drop platzierbar
 - Wiederholte Pattern-Platzierungen referenzieren dasselbe Pattern (Bearbeitung an einer Stelle aktualisiert alle Vorkommen)
 - Datenmodell: Song = Liste von `{pattern_id, start_position}`-Einträgen, Patterns bleiben eigenständige, wiederverwendbare Bausteine
- ✓ **Entscheidung (30.09.2026):** Version 1.0 arbeitet mit **einem festen Tempo und einer Taktart je Projekt**; Tempo- und Taktartwechsel innerhalb eines Songs sind eine spätere Erweiterung

3.5 Backing-Track (Referenz-Wiedergabe und Aufnahme)

- ✓ **Entscheidung:** Import eines Songs als WAV-Datei, synchrone Hintergrund-Wiedergabe während der Live-Drum-Eingabe
- ✓ **Entscheidung (30.09.2026): Aufnahme der Live-Eingabe, während der Backing-Track läuft**, ist Teil von Version 1.0 (nicht MVP)
 - Aufnahme im Song-Modus ab beliebiger Song-Position
 - Jeder Durchgang wird als neues Pattern „Take n“ als Block auf die Drums-Spur gelegt; bestehende (wiederverwendete) Patterns werden nicht verändert
 - Aufgenommen werden nur Drum-Noten, der Backing-Track selbst bleibt unverändert

- ☐ Backing-Track und Live-Sample-Wiedergabe laufen über denselben Transport/Clock (gemeinsamer Play/Stop, kein Drift)
- ☐ Mischung von Backing-Track- und Live-Drum-Buffer im Audio-Callback (Gain-Kontrolle)
- ☐ Wellenform-Anzeige des Backing-Tracks in der Timeline zur Orientierung (JUCE `AudioThumbnail`)

3.6 MIDI Import/Export

- ☐ Import von Standard-MIDI-Files (.mid) in den Pattern-Editor
- ☐ Export von Patterns als Standard-MIDI-File
- ☐ Song-Export: komplette Timeline (alle verketteten Patterns) zu einer durchgehenden MIDI-Datei zusammenrendern

3.7 Audio-Ausgabe (Plattformabhängig)

- ☐ Linux: ALSA-Unterstützung
- ☐ Linux: JACK-Unterstützung
- ☐ Windows: ASIO-Unterstützung (Hinweis: ASIO-SDK selbst nicht Open Source, muss separat eingebunden werden, siehe Abschnitt 5)

3.8 Projektverwaltung

- ☒ **Entscheidung (30.09.2026):** Speichern und Laden von Projekten als eigene Projektdatei (**.dpp**) ist Teil des MVP
 - Eine .dpp-Datei enthält Kit (Sample-Zuordnung), Patterns, Song-Arrangement, Backing-Track-Verweis, Tempo, Taktart und Mix-Einstellungen
 - Sample- und Backing-Track-Dateien werden relativ zur .dpp referenziert, damit Projekte portabel bleiben
 - Fehlende Sample-Dateien werden beim Laden gemeldet, das Projekt lädt trotzdem
- ☐ Neu, Öffnen, Speichern, Speichern unter; Nachfrage bei ungespeicherten Änderungen
- ☒ **Entscheidung (30.09.2026): Undo/Redo** (Strg+Z / Strg+Y) für alle Editier-Aktionen ist Teil des MVP
 - Jeder Aufnahme-Durchgang ist ein Undo-Schritt, damit ein missglückter Take sofort verworfen werden kann
 - Umsetzung über JUCE `ValueTree` + `UndoManager`

4. Nicht-funktionale Anforderungen

- **Latenz:** Zielwert **5 ms** Round-Trip-Latenz bei geeigneter Hardware/Treiber (ASIO/JACK, dediziertes Audio-Interface); 5 ms entsprechen ca. 220–240 Samples **Gesamtlatenz** bei 44,1/48 kHz (Eingang + Ausgang + Treiber), nicht der Puffergröße. **Entscheidung (30.09.2026):** Zielkonfiguration ist eine Puffergröße von **128 Samples bei 48 kHz** (2,7 ms je Puffer); Anschlag bis Ton ≤ 6 ms, Ziel 5 ms. Kein Hard-Garantiewert auf jedem System (Onboard-Sound/hohe Systemlast können höhere Werte erzwingen) — Buffer-Size in den Audio-Settings einstellbar, damit Nutzer bei Bedarf hochsetzen können.
- **Stabilität:** kein Audio-Glitching/Dropouts im Normalbetrieb
- **Plattformen:** Linux (primäre Entwicklungsumgebung, Linux Mint) und Windows (Cross-Build via CI, siehe Abschnitt 5)
- **Lizenz:** ausschließlich Open-Source-Bibliotheken (JUCE selbst AGPLv3)

- **Bedienbarkeit:** Einsteiger-/Hobbyanwender-tauglich, keine professionelle DAW-Komplexität nötig
- **Entwicklungsvorgaben (Entscheidung 03.10.2026):**
 - Testgetriebene Entwicklung (TDD, Red-Green-Refactor)
 - 100 % Unit-Test-Abdeckung (Zeilen und Branches) der Logik; nicht testbarer Glue-Code (Zeichenroutinen, Treiber-Callbacks) nach dem Humble-Object-Prinzip minimal halten und begründet ausnehmen
 - Clean Code (sprechende Namen, SOLID, kleine Funktionen), geprüft mit clang-format/clang-tidy, Warnungen als Fehler
 - Inversion of Control mit Constructor Dependency Injection, keine Singletons, eine zentrale Composition Root
 - Soweit die Leistungskriterien (Latenz, Echtzeit-Sicherheit) es erlauben: im Audio-Callback wird DI statisch über Template-Parameter aufgelöst statt über virtuelle Aufrufe pro Sample

5. Technische Rahmenbedingungen / bekannte Einschränkungen

- **ASIO-SDK:** nicht Open Source, Lizenzbedingungen von Steinberg erlauben kompilierte Distribution, aber keine Weitergabe des SDK-Quellcodes im Repository. SDK muss lokal/in CI separat eingebunden werden.
- **Windows-Build unter Linux Mint:** empfohlener Weg ist eine CI-Pipeline (z. B. GitHub Actions, windows-latest-Runner mit MSVC), da MinGW-Cross-Compiling insbesondere mit dem ASIO-SDK erfahrungsgemäß Kompatibilitätsprobleme verursacht.
- **Threading:** striktes Trennen von Audio-Thread und GUI-Thread erforderlich (JUCE-typisch), potenzielle Fehlerquelle für Race Conditions — Debugging-Aufwand einplanen.

6. Grobe Meilensteine / Roadmap

Phase	Inhalt	Geschätzter Aufwand
1	JUCE/CMake-Setup, erstes lauffähiges Audio-Callback	1 Woche
2	MIDI-Input (Tastatur + Hardware)	1-1,5 Wochen
3	Sample-Engine (mehrere Samples, Polyphonie)	1,5-2 Wochen
4	Pattern-Editor GUI mit Maus-Editierung	3-5 Wochen
5	Song-Arrangement (Timeline, Pattern-Verkettung)	1,5-2,5 Wochen
6	Backing-Track-Import/-Wiedergabe (WAV, synchron zur Live-Eingabe)	1-1,5 Wochen
7	MIDI-Import/Export (inkl. Song-Zusammenrendern)	1-1,5 Wochen
8	Audio-Ausgabe ALSA/JACK/ASIO, Cross-Platform-Tests	1,5-2,5 Wochen
9	Bugfixing, Feinschliff	2-4 Wochen

MVP-Ziel (Samples abspielen, Live-Aufnahme ins Pattern, Pattern-Editor, Projektdatei, MIDI-Export): ca. 16 Wochen

Vollversion 1.0 (Song-Arrangement, Backing-Track mit Aufnahme, MIDI-Import, stabile Multi-Plattform-Ausgabe): ca. 28 Wochen

Die Phasen oben sind die ursprüngliche Grobschätzung. Die verbindliche Planung mit 12 Arbeitspaketen (242 Std., 290 Std. inkl. Puffer) steht im Pflichtenheft. Sie ergänzt eigene Pakete für Datenmodell/Projektdatei und Transport/Aufnahme und zieht den Windows-CI-Build nach vorne.

7. Offene Fragen (zur weiteren Planung)

- ✓ ~~Welches Pattern-Format bevorzugt: klassischer Step-Sequencer (Grid) oder freie Piano-Roll-Ansicht?~~ → **Entschieden:** Piano-Roll mit Snap-to-Grid (siehe 3.3)
 - ✓ ~~Wie viele Sample Slots/Pads mindestens vorgesehen (z. B. 8, 16, mehr)?~~ → **Entschieden:** Standard-Kit nach GM-Drum-Map (ca. 16–20 Kernslots: Kick, Snare, 3× Tom, Hi-Hat closed/open/pedal, 2× Crash, Ride, Ride Bell, Cowbell, Clap, Side Stick, Tambourine), siehe 3.1
 - ✓ ~~Soll es Pattern-Ketten/Songs (mehrere Patterns verkettet) geben, oder erstmal nur Einzel-Patterns?~~ → **Entschieden:** Ja, Song-Arrangement ist Ziel (Timeline mit Pattern-Verkettung), siehe 3.4
 - ✓ ~~Zielwert für Audio-Latenz?~~ → **Entschieden:** 5 ms bei geeigneter Hardware, einstellbare Buffer-Size als Fallback (siehe Abschnitt 4)
 - ✓ ~~Soll Quantisierung/Swing-Funktion Teil des MVP sein oder spätere Phase?~~ → **Entschieden:** Spätere Phase; MVP-Fokus liegt auf manueller Eingabe via Tastatur/MIDI-Hardware (siehe 3.2)
 - ✓ ~~Sample-Formate: nur WAV oder auch weitere (z. B. AIFF, FLAC)?~~ → **Entschieden:** MVP nur WAV, Erweiterbarkeit auf AIFF/FLAC architektonisch offengehalten (siehe 3.1)
 - ✓ ~~Soll die Live-Eingabe aufgenommen werden können?~~ → **Entschieden:** Ja, Kernfunktion: Aufnahme ins Pattern im MVP, Aufnahme zum laufenden Backing-Track in Version 1.0 (siehe 3.2, 3.5)
 - ✓ ~~Zeitauflösung des Datenmodells?~~ → **Entschieden:** 960 PPQ wie in Reaper (siehe 3.3)
 - ✓ ~~Sollen Projekte gespeichert und geladen werden können?~~ → **Entschieden:** Ja, eigene Projektdatei .dpp im MVP (siehe 3.8)
 - ✓ ~~Pitch-Regler je Sample-Slot trotz Ausschluss von Effekten?~~ → **Entschieden:** Ja, als Tonhöhenänderung über Abspielgeschwindigkeit, Priorität Soll (siehe 3.1)
 - ✓ ~~Undo/Redo im MVP?~~ → **Entschieden:** Ja, für alle Editier-Aktionen und Aufnahme-Durchgänge (siehe 3.8)
 - ✓ ~~Tasten-Mapping bei deutscher Tastatur?~~ → **Entschieden:** Scancodes, im GUI frei einstellbar (siehe 3.2)
 - ✓ ~~Welche Puffergröße ergibt das Latenzziel?~~ → **Entschieden:** 128 Samples bei 48 kHz (siehe Abschnitt 4)
 - ✓ ~~Tempowechsel im Song?~~ → **Entschieden:** Nicht in v1.0, festes Tempo je Projekt (siehe 3.4)
-

8. Out of Scope (vorerst)

- VST/Plugin-Hosting (reine Standalone-Anwendung)
- Mixing/Effekte-Verarbeitung über Basis-Sample-Wiedergabe hinaus (Lautstärke und Pitch je Slot sowie die Mix-Regler zählen zur Basis-Wiedergabe)
- Cloud-Sync/Kollaborationsfunktionen
- macOS-Unterstützung (nicht explizit gefordert, ggf. spätere Erweiterung da JUCE es technisch ermöglicht)

9. Status: alle offenen Fragen entschieden ✓

Alle Punkte aus Abschnitt 7 sind geklärt. Das Pflichtenheft (Stand 30.09.2026) setzt dieses Lastenheft um. Nächster sinnvoller Schritt: AP0 (JUICE/CMake-Setup, erstes lauffähiges Audio-Callback, Windows-CI-Build) starten.

Dieses Dokument dient als lebende Planungsgrundlage und wird im Projektverlauf gemeinsam weiterentwickelt/angepasst.